Laboratory Assignment 0: Parallel Execution Environment

Jose Ricardo Arias Perez
jose.ricardo.arias@estudiantat.upc.edu

Group: cpds1118
Course 2026/27, Q1

Concurrency, Parallelism and Distributed Systems (CPDS)
FIB – Universitat Politècnica de Catalunya

October 4th, 2026

1 Node Architecture and Memory

1.1 Architectural characteristics (Question 1)

Characteristic	Value
NUMA nodes per node	2
Sockets per node	2
Cores per socket	16
Threads per core	2
Maximum core frequency	3400 MHz
L1-I cache size (per core)	32 KB
L1-D cache size (per core)	48 KB
L2 cache size (per core)	1280 KB
Last-level cache (per socket)	24 MB
Main memory (per socket)	62.3 GB (NUMA node 0) / 61.0 GB (NUMA node 1)
Main memory (total)	123.3 GB

Table 1: Architecture of boada-18, from `lscpu` and `numactl`.

1.2 NUMA architecture (Question 3)

NUMA means Non-Uniform Memory Access: the access time depends on which socket owns the memory. **boada-18** has two sockets, and each socket has its own memory bank (62.3 GB and 61.0 GB). Any core can access any byte of the full 123 GB, but the path differs. When core 0 (socket 0) reads from the memory physically attached to socket 0, it goes directly through the local memory controller. When it reads from socket 1 memory, the request crosses the inter-socket link. That extra hop costs time.

The ACPI SLIT distances reported by `numactl -hardware` are shown in Table 2. The value 10 is the baseline (local access) and 20 is the cost of reaching the other socket's memory.

	Node 0	**Node 1**
Node 0	10	20
Node 1	20	10

Table 2: NUMA distance matrix obtained with `numactl` on boada-18.

The local-to-remote ratio is $10 : 20 =$ **1:2**: accessing memory on the other socket is twice as expensive as accessing local memory.

Machine (123GB total)

Package L#0

NUMANode L#0 P#0 (62GB)

L3 (24MB)

L2 (1280KB) | L2 (1280KB) | 16x total | L2 (1280KB)

L1d (48KB) | L1d (48KB) | L1d (48KB)

L1i (32KB) | L1i (32KB) | L1i (32KB)

Core L#0
- PU L#0 P#0
- PU L#1 P#32

Core L#1
- PU L#2 P#1
- PU L#3 P#33

Core L#15
- PU L#30 P#15
- PU L#31 P#47

PCI 00:11.5

PCI 00:17.0

0.6 — 0.6 — PCI 02:00.0

7.9 — 7.9 — PCI 17:00.0
- Block sdb 3725 GB
- Block sda 3725 GB

2.5 — 2.5 — PCI 31:00.0
- Net ens5f0

2.5 — PCI 31:00.1
- Net ens5f1

2.5 — PCI 31:00.2
- Net ens5f2

2.5 — PCI 31:00.3
- Net ens5f3

4.9 — 4.9 — PCI 4b:00.0
- Net ens1f0

4.9 — PCI 4b:00.1
- Net ens1f1

Package L#1

NUMANode L#1 P#1 (61GB)

L3 (24MB)

L2 (1280KB) | L2 (1280KB) | 16x total | L2 (1280KB)

L1d (48KB) | L1d (48KB) | L1d (48KB)

L1i (32KB) | L1i (32KB) | L1i (32KB)

Core L#16
- PU L#32 P#16
- PU L#33 P#48

Core L#17
- PU L#34 P#17
- PU L#35 P#49

Core L#31
- PU L#62 P#31
- PU L#63 P#63

Host: boada-18

Date: Fri 02 Oct 2026 05:13:36 PM CEST

2 Timing Sequential and Parallel Executions

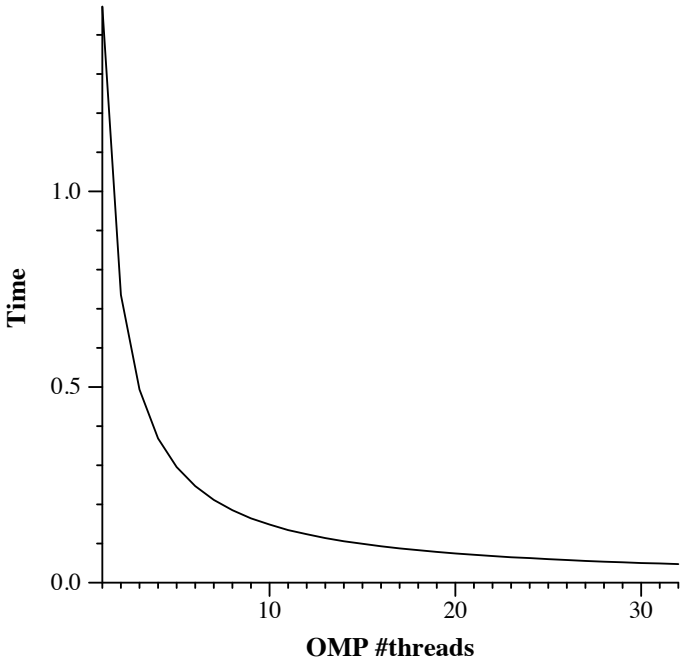
2.1 OpenMP scaling (Question 3)

Strong scaling keeps the problem size fixed (10^9 iterations) and increases the number of threads, so parallelism is used to reduce the execution time. The ideal speedup with p threads is p. Figure shows that the time drops from 1.47 s with 1 thread to 0.047 s with 32 threads, a speedup of 30.99 (efficiency 0.97).

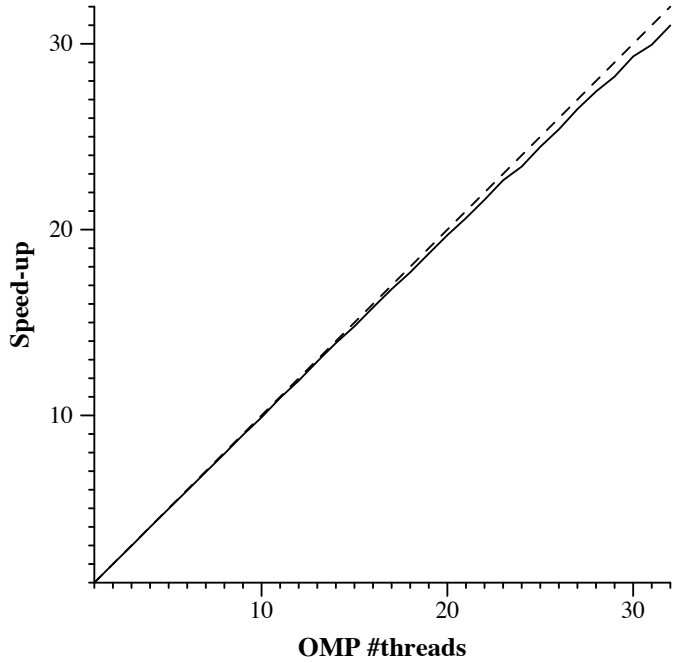
Weak scaling grows the problem size proportionally with the number of threads (10^9 iterations per thread), so parallelism is used to solve a larger problem in the same time. The ideal is a constant execution time (an efficiency of 1). Figure shows the efficiency stays between 0.99 and 1.00 for all thread counts from 1 to 32.

Beyond 32 threads: boada-18 has 32 physical cores, so threads 33–64 run on the second hardware thread of a core that is already busy. With 33 threads the time jumps from 0.0476 s to 0.0906 s and the speedup falls from 30.8 to 16.2. The two threads sharing a core each run at about half speed, and the parallel region ends only when the slowest thread finishes. As more threads are added, each thread's share of work shrinks, so the time decreases again, but with 64 threads (two per core) it is 0.0483 s, no better than the 0.0476 s obtained with 32 threads.

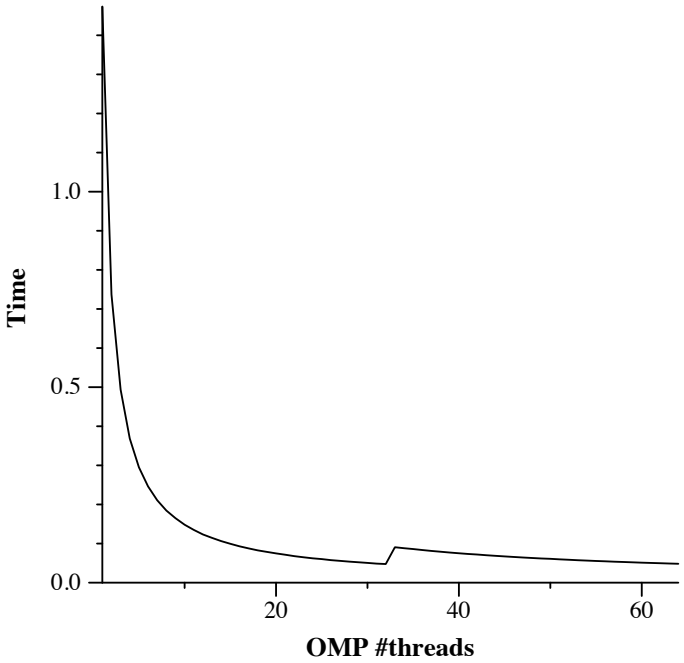
The reason **Hyper-Threading** gives no gain is that `pi_omp` is a **CPU-bound** application, not a memory-bound one. Hyper-Threading helps when a thread leaves the core idle while waiting for memory.



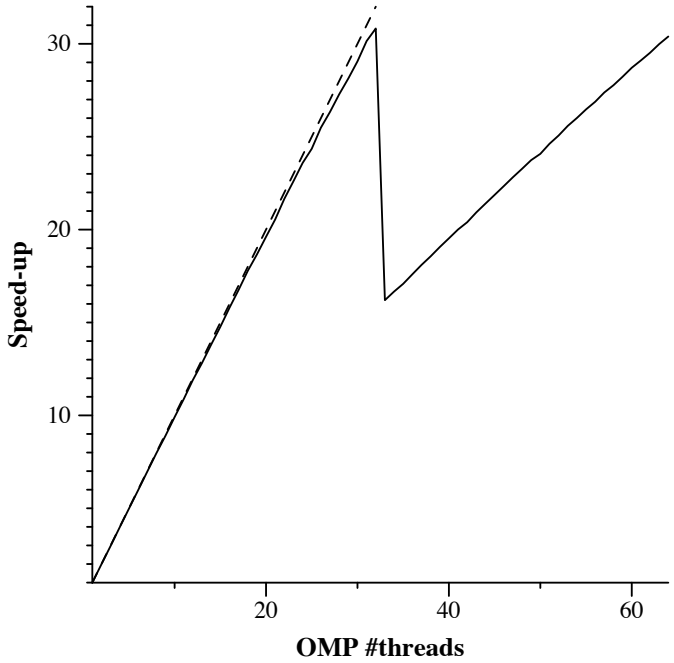
cpds1118
Min elapsed execution time
Fri Oct 2 07:47:33 PM CEST 2026



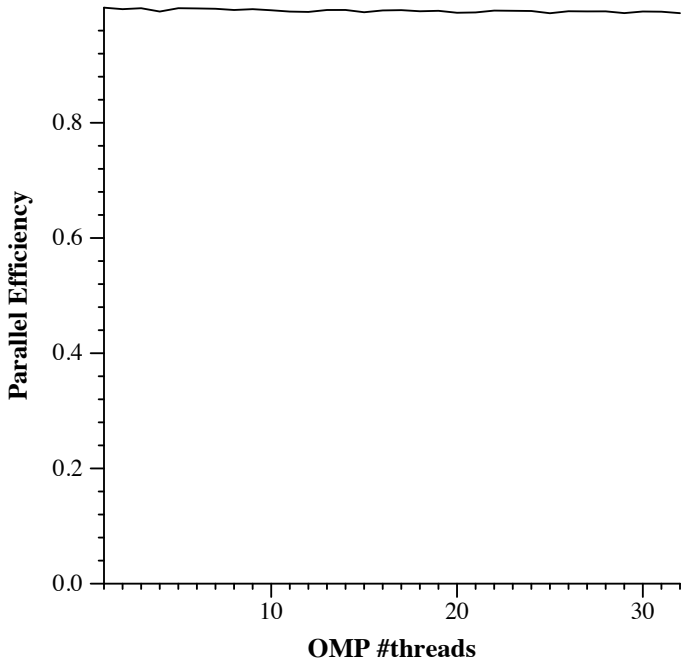
cpds1118
Speed-up wrt sequential time
Fri Oct 2 07:47:33 PM CEST 2026



cpds1118
Min elapsed execution time
Fri Oct 2 07:52:53 PM CEST 2026



cpds1118
Speed-up wrt sequential time
Fri Oct 2 07:52:53 PM CEST 2026



cpds1118

Parallel Efficiency w.r.t. one thread (weak scaling)

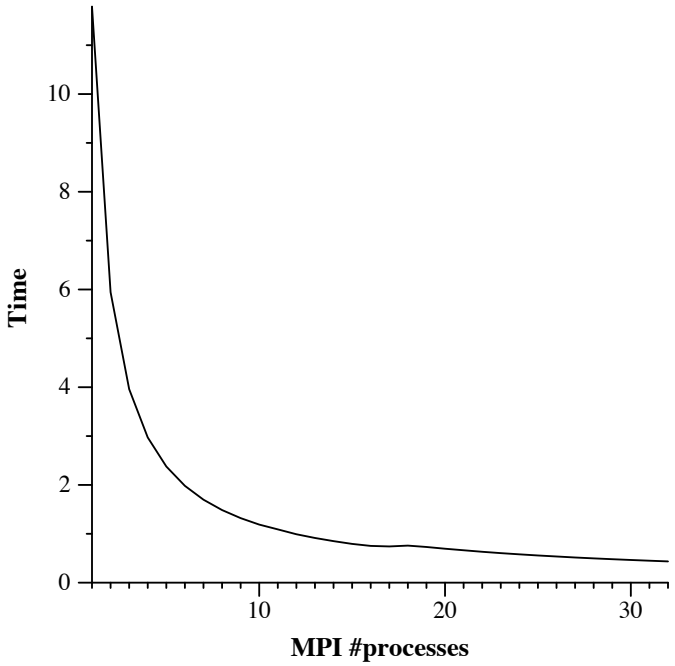
Fri Oct 2 07:48:13 PM CEST 2026

2.2 MPI and hybrid scaling (Question 4)

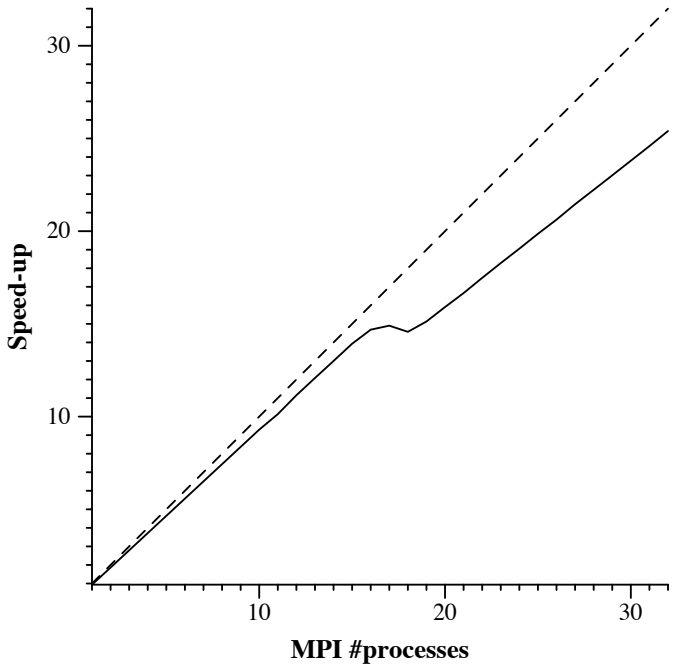
2.2.1 MPI strong scaling (1–32 processes)

Figure shows the execution time and speedup of the Monte Carlo code with 1 to 32 MPI processes. The execution scales well up to 16 processes and less well from 16 to 32. With 1 process the speedup is 0.937: the MPI program takes 11.79 s against 11.05 s for `pi_seq`, a 6.7% overhead. For OpenMP, the single-thread overhead was only 0.1% (speedup 0.999). MPI therefore introduces considerably more overhead than OpenMP, because it has to start processes and set up the communication layer, while OpenMP only creates threads inside one process.

Up to 16 processes the scaling is almost linear (each gets its own core): 16 processes take 0.752 s, a speedup of 14.69 over `pi_seq` and an efficiency of 0.98 relative to the 1-process MPI run. After that, from 16 to 32 processes, it improves only by a factor of 1.73 instead of 2. With 32 processes the time is 0.435 s, a speedup of 25.40 and an efficiency of 0.79. From the 17th process on, processes share cores through Hyper-Threading, the sharing spreads over more cores and the time decreases again, but two processes on one core do not run as fast as two processes on two cores. Communication is not the bottleneck: the program performs only one `MPI_Reduce` of a single value at the end.



cpds1118
Min elapsed execution time
Fri Oct 2 08:07:08 PM CEST 2026



cpds1118
Speed-up wrt sequential time
Fri Oct 2 08:07:08 PM CEST 2026

2.2.2 Hybrid MPI + OpenMP (2 processes, varying threads)

Table 3 shows the elapsed time of `pi_mpi_omp` with 2 MPI processes and 1 to 32 OpenMP threads per process. The default run uses two nodes (one process on boada-18 and one on boada-19). The single-node run was made with `#SBATCH -nodes=1`, and both processes ran on boada-18.

Total threads (2 MPI × threads)	Time, 2 nodes (s)	Time, 1 node (s)
2 (2 × 1)	5.930	6.930
4 (2 × 2)	2.971	3.466
8 (2 × 4)	1.490	1.734
16 (2 × 8)	0.751	0.867
24 (2 × 12)	0.497	0.578
32 (2 × 16)	0.373	0.434
40 (2 × 20)	0.299	0.348
48 (2 × 24)	0.249	0.292
56 (2 × 28)	0.213	0.249
64 (2 × 32)	0.187	0.219

Table 3: Elapsed time for 2 MPI processes with varying OpenMP threads per process (2^{30} iterations).

2.2.3 Summary of key execution times

Configuration	Time (s)	Speedup vs seq	Efficiency
Sequential (`pi_seq`)	11.05	1.00	—
32 MPI processes	0.435	25.40	0.79
2 MPI × 32 threads (2 nodes)	0.187	59.07	0.92
2 MPI × 32 threads (1 node)	0.219	50.53	0.79

Table 4: Comparison of key configurations (2^{30} iterations). Efficiency is speedup divided by the number of processes × threads.

32 MPI processes vs. 2 MPI × 16 threads. With the same 32 workers, 32 MPI processes (0.435 s) are slower than 2 processes with 16 threads each on two nodes (0.373 s). The difference comes from the second node, not from MPI versus OpenMP: on a single node, 2 × 16 threads take 0.434 s, the same as 32 MPI processes. Spreading the work over two nodes gives every worker more hardware resources.

Speedup of 2 MPI × 32 threads. On two nodes this configuration takes 0.187 s, a speedup of 11.05/0.187 = 59.07 over the sequential Monte Carlo code (efficiency 0.92 on 64 threads).

Comparison with OpenMP at 64 threads. The result does not match Section 1.4.3. There, going from 32 to 64 OpenMP threads gave no improvement (0.0476 s to 0.0483 s). Here, going from 2 × 16 to 2 × 32 threads halves the time, both on two nodes (0.373 to 0.187 s, ×2.00) and on a single node (0.434 to 0.219 s, ×1.98).

The single-node run proves that the improvement is not due to having more physical cores: on one node, 64 threads share 32 physical cores through Hyper-Threading, exactly as in the OpenMP 64-thread test, and the time still halves. The different behaviour comes from the code. `pi_omp` keeps each core's floating-point units fully busy with one thread, so a second hardware thread has nothing to add. The Monte Carlo code is limited by latency instead: each call to

`my_rand` depends on the seed produced by the previous call, through a 64-bit multiplication and modulo. One thread spends most cycles waiting for the previous result, leaving the core's execution units idle, and the second hardware thread fills those idle cycles. Hyper-Threading therefore nearly doubles throughput for this code but not for `pi_omp`.

Using one node instead of two makes every configuration about 17% slower (ratio 1.16–1.17 in all rows of Table 3, including 2×1), so the 1-node 2×32 run takes 0.219 s instead of 0.187 s.